

Section 7

Week 8 — Recovery I: WAL, Crash Recovery & Backups

Agenda

- 1. The Durability Problem — what does it mean for data to survive?
 - 2. Write-Ahead Logging (WAL) — the core mechanism behind durability
 - 3. Crash Recovery — what happens when PostgreSQL crashes mid-transaction?
 - 4. Checkpoints — how PostgreSQL manages WAL files
 - 5. Proving Durability — transactions that survive failure
 - 6. Logical Backup with `pg_dump` — exporting your database
 - 7. Restore with `psql` / `pg_restore` — bringing it back
 - 8. Practice Lab — full backup & restore cycle
-

From Last Week

We focused on making queries faster — selectivity, partial indexes, expression indexes, covering indexes, and common indexing mistakes.

This week is different. We're not about speed anymore — we're about **safety**. What happens when the power goes out? When the server crashes in the middle of an UPDATE? How does PostgreSQL make sure your committed data is never lost?

1. The Durability Problem

ACID has four properties. Quick recap:

Property	Meaning
Atomicity	All or nothing — a transaction fully completes or fully rolls back
Consistency	The database moves from one valid state to another
Isolation	Concurrent transactions don't interfere with each other
Durability	Once committed, the data survives any failure

We already covered A, C, and I in Weeks 3–5. Today is all about **D — Durability**.

The problem

Databases don't write directly to disk every time you run an INSERT or UPDATE — that would be extremely slow. Instead, changes are made in **memory (RAM)** first, inside a shared area called the **buffer cache** (also called shared buffers).

```
Client runs: UPDATE customer SET email = 'new@email.com' WHERE customer_id = 1;
```

What happens:

1. PostgreSQL finds the page in memory (or loads it from disk)
2. Modifies the row IN MEMORY
3. Returns "UPDATE 1" to the client

The problem: **RAM is volatile**. If the server crashes right now, that change is gone — the disk still has the old email.

So how does PostgreSQL guarantee durability? The answer is **WAL**.

2. Write-Ahead Logging (WAL)

The Rule

Before any change reaches disk, a log record describing that change must be written to disk FIRST.

This is the Write-Ahead Logging rule. It's the single most important concept in database recovery.

How it works — step by step

```
Client: UPDATE customer SET email = 'new@email.com' WHERE customer_id = 1;
```

```
Step 1: PostgreSQL modifies the row in MEMORY (buffer cache)
```

```
Step 2: PostgreSQL writes a WAL record to the WAL file ON DISK
```

```
      → "Changed row X in table customer from 'old@email.com' to  
'new@email.com'"
```

```
Step 3: Returns "UPDATE 1" to the client
```

```
The actual table file on disk? NOT updated yet. It will be updated later.
```

Why this works

- Writing to WAL is **sequential** (append-only) → very fast
- Writing to the actual table file is **random** (find the right page, update it) → slow
- WAL gives us durability WITHOUT the performance cost of writing every change to the table immediately

What's inside a WAL record?

Each WAL record contains enough information to **redo** the change:

Field	Meaning
Transaction ID	Which transaction made this change
Table / Page	Which table and which page was modified
Before image	What the data looked like before (for undo)
After image	What the data should look like after (for redo)
Timestamp	When the change happened

Think of WAL as a **receipt** — even if the actual product (data file) is lost or damaged, the receipt proves what should be there, and you can reconstruct it.

The COMMIT moment — when is data truly safe?

```
BEGIN;  
  
UPDATE customer SET email = 'new@email.com' WHERE customer_id = 1;  
  
UPDATE customer SET last_name = 'Johnson' WHERE customer_id = 1;  
  
COMMIT;
```

Here's the critical sequence:

```
1. UPDATE email      → WAL record written to disk  
  
2. UPDATE last_name → WAL record written to disk  
  
3. COMMIT            → COMMIT WAL record written to disk  
  
                    → PostgreSQL returns "COMMIT" to client
```

The COMMIT is durable the moment the COMMIT WAL record hits disk. Not when the table files are updated — that happens later in the background.

If the server crashes:

- **After** the COMMIT WAL record is on disk → data is safe, will be recovered
- **Before** the COMMIT WAL record is on disk → transaction is rolled back, as if it never happened

Either way, the database stays consistent.

Demo — Exploring WAL

What is `pg_current_wal_lsn()` ?

LSN stands for **Log Sequence Number**. Think of it as a **page number in the WAL notebook**. Every time PostgreSQL writes a new WAL record, the LSN moves forward — like turning to the next page.

```
-- See the current "page number" in the WAL
```

```
SELECT pg_current_wal_lsn();
```

```
-- Example output: 0/1A3B5C8
```

The output `0/1A3B5C8` is a position in the WAL stream. The exact value doesn't matter — what matters is watching it **change** after a transaction:

```
-- Step 1: Record current position
```

```
SELECT pg_current_wal_lsn() AS before;
```

```
-- Step 2: Run a transaction
```

```
BEGIN;
```

```
UPDATE customer SET last_name = 'WAL-TEST' WHERE customer_id = 1;
```

```
COMMIT;
```

```
-- Step 3: Check position again - it moved forward!
```

```
SELECT pg_current_wal_lsn() AS after;
```

The LSN increased → that proves WAL records were written to disk for our transaction.

Practice — Part A

Run these queries to observe how WAL works in real time. Watch how the LSN changes after different operations.

```
-- 1. Record the current WAL position
```

```
SELECT pg_current_wal_lsn() AS before_position;
```

```
-- 2. Run a small transaction
```

```
BEGIN;
```

```
INSERT INTO customer (store_id, first_name, last_name,  
email, address_id, activebool, create_date)
```

```
VALUES (1, 'WAL', 'Test', 'wal.test@example.com', 1, true, CURRENT_DATE);
```

```
COMMIT;

-- 3. Record the WAL position again

SELECT pg_current_wal_lsn() AS after_position;

-- 4. Clean up

DELETE FROM customer WHERE email = 'wal.test@example.com';
```

Questions:

- Did the LSN increase? → Yes, because WAL records were written for the INSERT and COMMIT.
- If we had done ROLLBACK instead of COMMIT, would the LSN still increase? → Yes — WAL records are written during the transaction, not just at commit. ROLLBACK adds its own WAL record too.

Common Questions — WAL

Q: Why not just write directly to the data file on disk instead of writing to WAL first? Isn't WAL just extra work?

Great question — it seems like WAL is writing twice (once to WAL, once to the data file later). But the key difference is **how** the writes happen:

- **WAL writes are sequential** — PostgreSQL appends to the end of the WAL file, one record after another. This is like writing a list on paper from top to bottom. Sequential disk writes are extremely fast.
- **Data file writes are random** — the customer table page could be at position 500 on disk, the rental page at position 12,000. Each change jumps to a different location. Random disk I/O is 10–100× slower than sequential.

If PostgreSQL wrote every change directly to the data file, a single transaction with 5 UPDATES across different tables would need 5 random disk seeks. With WAL, it's 5 sequential appends to one file — much faster.

Also, WAL is the **minimum** needed for durability. The data files get updated later in bulk during checkpoints, which is more efficient than doing it one row at a time.

Q: Does WAL make the database slower since we're writing twice?

Actually the opposite — WAL makes it **faster**. Without WAL, every COMMIT would have to flush the actual data pages to disk (random I/O, slow). With WAL, COMMIT only needs to flush the WAL record (sequential I/O, fast). The data pages are written lazily in the background at checkpoint time.

Q: Can I turn off WAL to make the database faster?

No. WAL is not optional in PostgreSQL — it's fundamental to how the database works. Without WAL, there's no crash recovery and no durability guarantee. You would lose data on every crash.

3. Crash Recovery

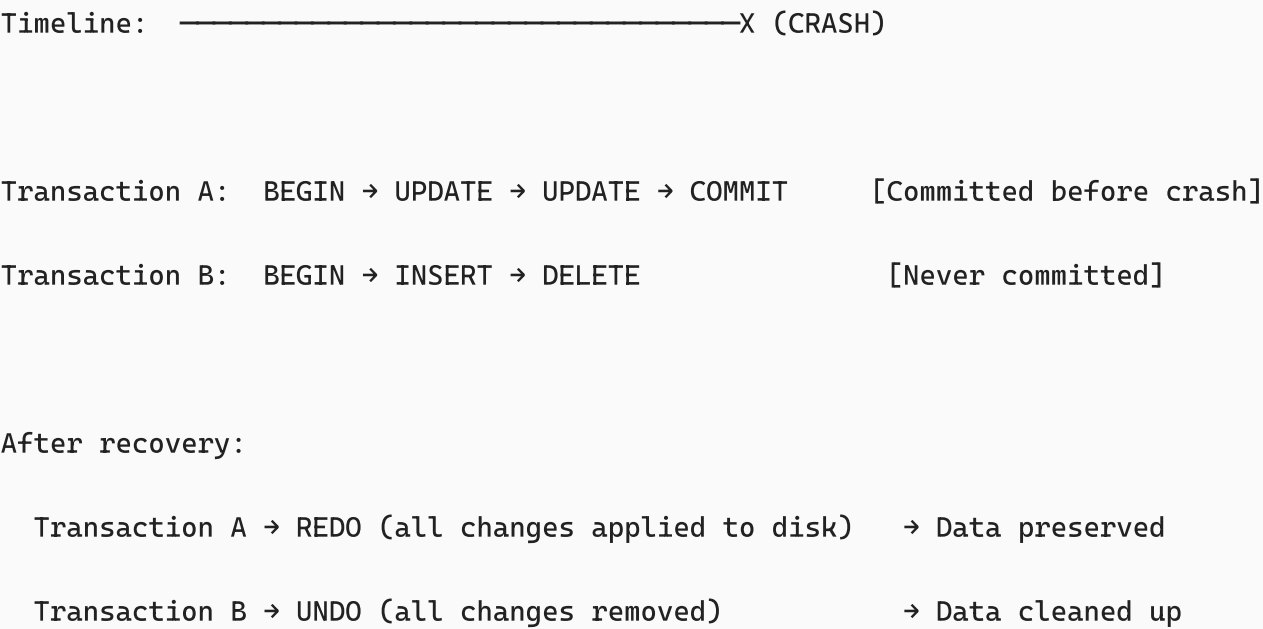
What happens when PostgreSQL starts after a crash?

When PostgreSQL restarts, it doesn't know if the last shutdown was clean or a crash. So it **always** checks WAL and does recovery if needed.

PostgreSQL startup sequence:

- 1. Read the WAL from the last checkpoint
- 2. REDO: Replay all WAL records for COMMITTED transactions
 - Applies changes that were in WAL but not yet on disk
- 3. UNDO: Roll back any transactions that were NOT committed
 - Removes partial changes from uncommitted transactions
- 4. Database is now consistent → ready to accept connections

Visual timeline



Why this is automatic

You don't run any command. You don't call any recovery procedure. PostgreSQL does this **automatically** every time it starts. For a DBA, the database just "comes back" after a crash — that's WAL doing its job.

How long does recovery take?

It depends on how much WAL needs to be replayed — which depends on **checkpoints** (next topic). More frequent checkpoints = less WAL to replay = faster recovery. But more frequent checkpoints = more disk I/O during normal operation.

Crash Simulation Demo

We will simulate a real crash by killing PostgreSQL without a clean shutdown, then watch it recover automatically.

Step 0: Find your data directory and show WAL files

Run this in **pgAdmin's Query Tool** (or any SQL query window):

```
SHOW data_directory;

-- Example output: C:\Program Files\PostgreSQL\18\data
```

Now open **PowerShell** or **File Explorer** and navigate to that path. The WAL files are inside the `pg_wal` subfolder:

```
● ● ●

dir "C:\Program Files\PostgreSQL\18\data\pg_wal"
```

WAL is not abstract — these are actual files sitting on disk. Each file is a 16MB segment.

```
-- In psql: see which WAL file is currently being written to

SELECT pg_walfile_name(pg_current_wal_lsn()) AS current_wal_file;
```

Step 1: Create test data in pgAdmin

Query Tab 1 — this transaction WILL commit:

```
-- Insert a row and COMMIT it - this MUST survive the crash

BEGIN;

INSERT INTO
  customer (store_id, first_name, last_name, email,
            address_id, activebool, create_date)

VALUES (1, 'CRASH', 'Survivor', 'crash.survivor@test.com', 1, true, CURRENT_DATE)

COMMIT;

-- Verify it's there

SELECT * FROM customer WHERE email = 'crash.survivor@test.com';

-- Row exists - committed and saved in WAL on disk
```

Query Tab 2 (open a NEW query tab in pgAdmin) — this transaction will NOT commit:

```
-- Insert a row but DO NOT commit – leave the transaction open

BEGIN;

INSERT INTO customer
(store_id, first_name, last_name, email,
 address_id, activebool, create_date)

VALUES (1, 'CRASH', 'Ghost', 'crash.ghost@test.com', 1, true, CURRENT_DATE);


-- DO NOT click Run on COMMIT – leave it hanging!

-- This simulates a transaction that was in progress when the crash happened
```

Step 2: Kill PostgreSQL (simulate crash)

Open **PowerShell as Administrator** (right-click → Run as Administrator) and run:

```
● ● ●

& "C:\Program Files\PostgreSQL\18\bin\pg_ctl"
stop -m immediate -D "C:\Program Files\PostgreSQL\18\data"
```

Mac / Linux — find data directory first with `SHOW data_directory;` in pgAdmin, then:

```
● ● ●

# Mac (Homebrew)

pg_ctl stop -m immediate -D /opt/homebrew/var/postgresql@18


# Linux (Ubuntu/Debian)

sudo -u postgres pg_ctl stop -m immediate -D /var/lib/postgresql/18/main
```

This kills PostgreSQL instantly — no checkpoint, no cleanup, like pulling the power cord.

pgAdmin will show connection errors — that's expected. The server is dead.

Step 3: Start PostgreSQL again

Windows (same Administrator PowerShell):

```
● ● ●

net start postgresql-x64-18
```

Mac / Linux:



```
# Mac (Homebrew)
```

```
brew services start postgresql@18
```

```
# Linux (Ubuntu/Debian)
```

```
sudo systemctl start postgresql
```

PostgreSQL starts and **automatically recovers** — no manual action needed.

You can check the log file at `C:\Program Files\PostgreSQL\18\data\log\` to see recovery messages:

```
LOG:  database system was not properly shut down; automatic recovery in progress
```

```
LOG:  redo starts at ...
```

```
LOG:  redo done at ...
```

```
LOG:  database system is ready to accept connections
```

Step 4: Check the results

Reconnect in pgAdmin (click the server, it will reconnect), then run:

```
-- The COMMITTED row survived the crash
```

```
SELECT * FROM customer WHERE email = 'crash.survivor@test.com';
```

```
-- → Row exists!
```

```
-- The UNCOMMITTED row is gone
```

```
SELECT * FROM customer WHERE email = 'crash.ghost@test.com';
```

```
-- → No rows returned – crash recovery undid it
```

```
-- Clean up
```

```
DELETE FROM customer WHERE email = 'crash.survivor@test.com';
```

This is the power of WAL:

- `crash.survivor@test.com` was **committed** → WAL had the COMMIT record → **survived**

- `crash.ghost@test.com` was **never committed** → crash recovery rolled it back → **gone**

Key point: Nobody ran any recovery command. PostgreSQL did it all automatically on startup.

Common Questions — Crash Recovery

Q: What if the WAL files themselves get corrupted or deleted?

Then PostgreSQL **cannot recover**. WAL is the only source of truth for recent changes. This is why production systems store WAL on reliable storage (RAID, SSDs) and archive WAL files to a separate location (WAL archiving — Week 9). If both the data files and WAL are lost, you fall back to the last `pg_dump` backup.

Q: Does crash recovery take a long time?

It depends on how much WAL has accumulated since the last checkpoint. If checkpoints happen every 5 minutes (default), recovery replays at most ~5 minutes of WAL — usually a few seconds. If checkpoints are very infrequent, recovery could take minutes.

Q: Do I need to do anything as a DBA after a crash?

No. Recovery is 100% automatic. PostgreSQL detects the crash on startup, replays WAL, and opens for connections. You might want to check the PostgreSQL logs to understand what caused the crash, but recovery itself requires no manual action.

4. Checkpoints

The problem WAL creates

WAL files grow over time. If PostgreSQL never wrote the actual data to disk, crash recovery would need to replay ALL WAL since the database was created — that could take hours.

What a checkpoint does

A **checkpoint** is when PostgreSQL:

1. Writes all **dirty pages** (modified data in memory) to the actual table files on disk
2. Records "everything up to this point is safely on disk" in a checkpoint record
3. Old WAL files before the checkpoint can now be recycled

WAL timeline:

[====old WAL====][====old WAL====][CHECKPOINT][===new WAL===][===new WAL===]

↑

Everything before this

is safely on disk.

Recovery starts HERE.

When do checkpoints happen?

Trigger	Description
<code>checkpoint_timeout</code>	Every 5 minutes by default
<code>max_wal_size</code>	When WAL files exceed this size (default 1GB)
Manual	<code>CHECKPOINT;</code> command (rarely needed)
Clean shutdown	PostgreSQL does a checkpoint before stopping

```
-- See checkpoint settings

SHOW checkpoint_timeout;

SHOW max_wal_size;
```

The trade-off

Frequent checkpoints	Infrequent checkpoints
+ Faster crash recovery	- Slower crash recovery
- More disk I/O during normal operation	+ Less disk I/O during normal operation

For most systems, the defaults are fine. Only production DBAs tune these settings.

Common Questions — Checkpoints

Q: If checkpoints write everything to disk, why do we still need WAL?

Because checkpoints don't happen on every COMMIT — they happen every 5 minutes (or when WAL fills up). Between checkpoints, the only proof of recent changes is in WAL. If the server crashes 3 minutes after a checkpoint, those 3 minutes of changes are only in WAL.

Q: What are "dirty pages"?

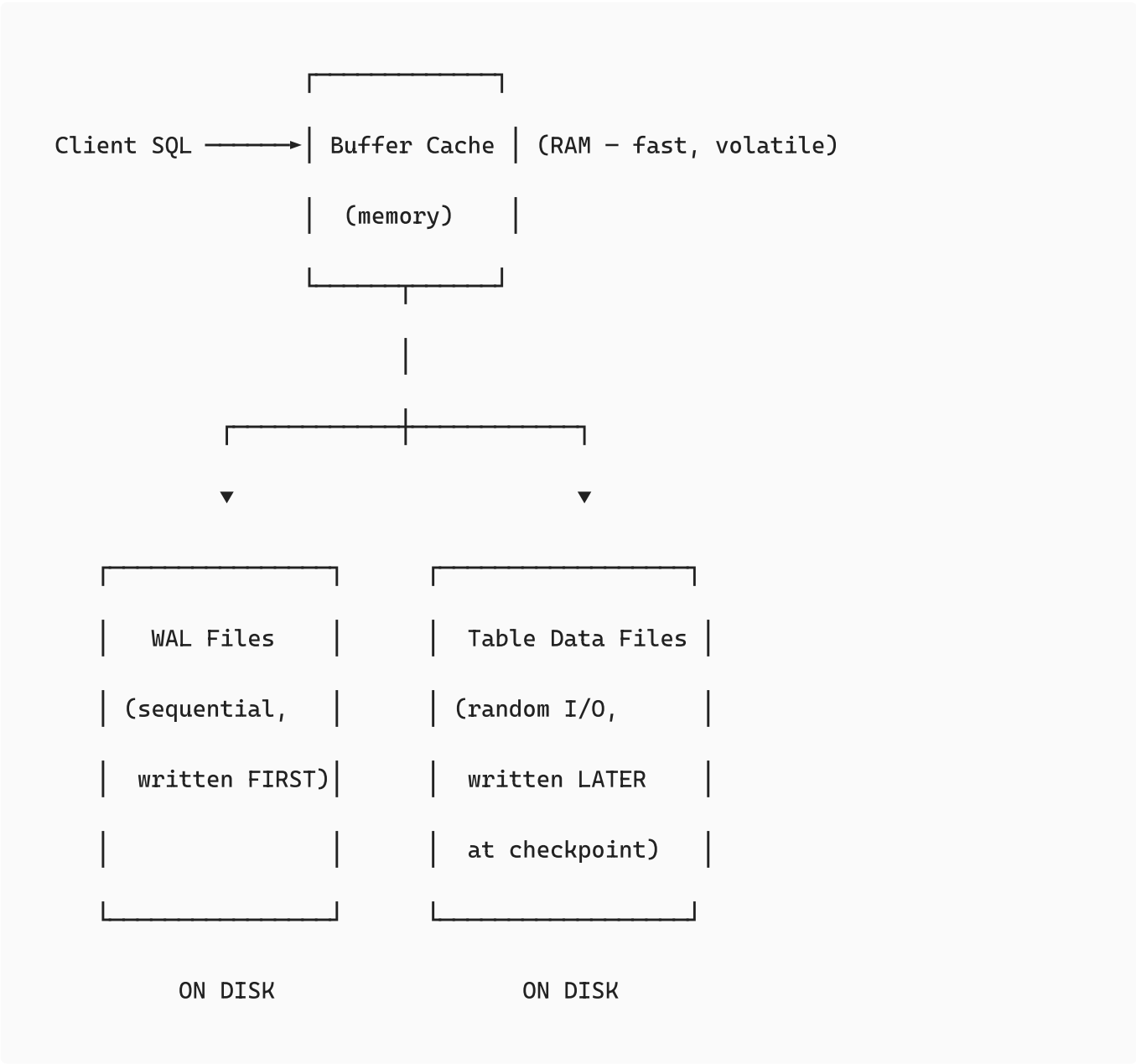
A "dirty page" is a data page in memory that has been modified but not yet written to the data file on disk. At checkpoint, all dirty pages are flushed to disk, making them "clean". After that, the WAL records that described those changes are no longer needed for recovery.

Q: Can I force a checkpoint manually?

Yes — run `CHECKPOINT;` in psql. But you rarely need to. PostgreSQL handles checkpoints automatically. The only common reason to force one is before taking a physical backup or before shutting down.

5. Summary So Far

Before moving to the practical backup/restore section, let's connect everything:



Concept	One-line summary
WAL	Log of every change, written to disk BEFORE the actual data
COMMIT	Durable when the COMMIT WAL record hits disk
Crash Recovery	Replay WAL to redo committed work, undo uncommitted work
Checkpoint	Flush dirty pages to disk so old WAL can be recycled

6. Logical Backup with pg_dump

Now we move from crash recovery (automatic) to **manual backups** — something a DBA does on purpose to have a copy of the database.

What is pg_dump ?

`pg_dump` is a command-line tool that creates a **logical backup** of a PostgreSQL database. It outputs SQL statements that can recreate the database from scratch.

Logical vs Physical backup

Type	What it saves	Tool	Use case
Logical	SQL statements (CREATE TABLE, INSERT, ...)	pg_dump	Single database, portable, can restore to different versions

Type	What it saves	Tool	Use case
Physical	Raw data files (binary copy of the whole cluster)	pg_basebackup	Full server backup, supports PITR (Week 9 topic)

Today we focus on **logical backup** with `pg_dump` .

pg_dump — Basic usage

First: open PowerShell and set the PATH so `pg_dump` and `psql` commands work:

● ● ●

```
$env:PATH = "C:\Program Files\PostgreSQL\18\bin;" + $env:PATH
```

If your PostgreSQL version is different, replace `18` with your version. Run `dir "C:\Program Files\PostgreSQL"` to check.

Now you can use all PostgreSQL tools in this terminal session:

● ● ●

```
# Basic dump of the entire dvdrental database to a SQL file

pg_dump -U postgres -f dvdrental_backup.sql dvdrental
```

Flag	Meaning
<code>-U postgres</code>	Connect as user <code>postgres</code>
<code>-f dvdrental_backup.sql</code>	Output file name
<code>dvdrental</code>	Database name to back up

Do NOT use `>` to redirect output in PowerShell — it saves in UTF-16 encoding which corrupts the backup file. Always use `-f` .

The output file contains all the SQL needed to recreate the database:

```
-- Inside dvdrental_backup.sql you'll see:

CREATE TABLE customer (

    customer_id integer NOT NULL,

    store_id smallint NOT NULL,

    ...

);

COPY customer (customer_id, store_id, first_name, ...) FROM stdin;

1 1 Mary Smith ...
```

```
2  1  Patricia  Johnson  ...  
  
...
```

pg_dump — Common options

```
● ● ●  
  
# Schema only (structure, no data)  
  
pg_dump -U postgres --schema-only -f dvdrental_schema.sql dvdrental  
  
# Data only (no CREATE TABLE statements)  
  
pg_dump -U postgres --data-only -f dvdrental_data.sql dvdrental  
  
# Dump a single table  
  
pg_dump -U postgres -t customer -f customer_only.sql dvdrental
```

Option	What it does
--schema-only	Only table definitions, indexes, constraints — no row data
--data-only	Only INSERT/COPY statements — no structure
-t table_name	Dump only one specific table

Demo — Running pg_dump

This demo is done in the terminal, not in psql.

```
● ● ●  
  
# Step 1: Full backup to SQL  
  
pg_dump -U postgres -f dvdrental_backup.sql dvdrental  
  
# Step 2: Check file size  
  
dir dvdrental_backup.sql  
  
# Step 3: Look at what's inside (first 50 lines)  
  
Get-Content dvdrental_backup.sql -TotalCount 50
```

```
# Step 4: Schema-only backup
```

```
pg_dump -U postgres --schema-only -f dvdrental_schema.sql dvdrental
```

```
# Step 5: Compare sizes - schema-only is much smaller
```

```
dir dvdrental_schema.sql
```

```
dir dvdrental_backup.sql
```

Discussion points:

- The full backup contains ALL data (INSERT/COPY statements for every row)
- The schema-only backup is just structure — useful for creating empty copies
- The SQL file is human-readable — you can open it and read exactly what it will do

Practice — Part C

Open a terminal and run these commands:

```
● ● ●
```

```
# 1. Create a full backup of dvdrental
```

```
pg_dump -U postgres -f my_backup.sql dvdrental
```

```
# 2. Create a schema-only backup
```

```
pg_dump -U postgres --schema-only -f my_schema.sql dvdrental
```

```
# 3. Create a backup of just the customer table
```

```
pg_dump -U postgres -t customer -f customer_only.sql dvdrental
```

```
# 4. Check file sizes - which is biggest? Why?
```

```
dir my_backup.sql
```

```
dir my_schema.sql
```

```
dir customer_only.sql
```

Answers:

- `my_backup.sql` is the largest — it contains all structure AND all data for every table.

- `my_schema.sql` is much smaller — only CREATE TABLE, INDEX, CONSTRAINT statements.
- `customer_backup.sql` is small — only one table's structure and data.
- **Why does this matter?** In production, full backups of large databases can be gigabytes. Knowing how to back up selectively saves time and storage.

Common Questions — pg_dump

Q: If pg_dump is running and someone modifies data at the same time, does the backup get corrupted?

No. `pg_dump` takes a **consistent snapshot** — it uses a transaction internally with `SERIALIZABLE` isolation (or `REPEATABLE READ` in newer versions). The backup reflects the database state at the moment `pg_dump` started, regardless of concurrent changes.

Q: Why can't I just copy the database folder instead of using pg_dump?

Because the database files might be in an inconsistent state — some pages are in memory (dirty pages), WAL hasn't been fully applied, etc. Copying raw files while PostgreSQL is running gives you a corrupted backup. `pg_dump` reads through the normal SQL interface, so it always gets consistent data. (For raw file backup, PostgreSQL has `pg_basebackup` which handles this correctly — Week 9.)

Q: Does pg_dump lock the database? Can users still work during a backup?

No locks on normal tables. `pg_dump` uses a snapshot, not locks. Users can continue reading and writing normally. The only exception: DDL operations like `DROP TABLE` or `ALTER TABLE` are blocked until `pg_dump` finishes reading that table.

7. Restore with psql / pg_restore

A backup is useless if you can't restore it. Let's do the full cycle.

Restoring a SQL backup (plain text)

For backups created with `pg_dump` in default format (`.sql` files):

Step 1: Create a new empty database — run this in **pgAdmin Query Tool** (connected to `postgres` database, not `dvdrental`):

```
CREATE DATABASE dvdrental_restored;
```

Step 2: Restore the backup — run this in **PowerShell**:

...

```
psql -U postgres -d dvdrental_restored -f dvdrental_backup.sql
```

Step 3: Verify in pgAdmin — connect to `dvdrental_restored` and run:

```
SELECT COUNT(*) FROM customer;
```

```
SELECT COUNT(*) FROM rental;
```



```
SELECT COUNT(*) FROM payment;
```

Demo — Full Backup & Restore Cycle

All commands in terminal:

In **pgAdmin** (connected to dvdrental):

```
-- 1. Make a change so we can verify the backup timing

UPDATE customer SET last_name = 'BACKUP-TEST' WHERE customer_id = 1;
```

In **PowerShell**:

```
● ● ●

# 2. Take the backup NOW (after the change)

pg_dump -U postgres -f dvdrental_full.sql dvdrental
```

In **pgAdmin** (connected to dvdrental):

```
-- 3. Make ANOTHER change AFTER the backup

UPDATE customer SET last_name = 'AFTER-BACKUP' WHERE customer_id = 1;
```

In **pgAdmin** (connected to postgres database):

```
-- 4. Create a new database

CREATE DATABASE dvdrental_test_restore;
```

In **PowerShell**:

```
● ● ●

# 5. Restore the backup

psql -U postgres -d dvdrental_test_restore -f dvdrental_full.sql
```

In **pgAdmin** (connect to dvdrental_test_restore):

```
-- 6. Check: which last_name does customer 1 have?

SELECT customer_id, last_name FROM customer WHERE customer_id = 1;
```

Expected result: Shows BACKUP-TEST — NOT AFTER-BACKUP . The backup is a **snapshot in time**. Changes made after pg_dump ran are not included.

Clean up (in pgAdmin, connected to postgres database):

```
DROP DATABASE IF EXISTS dvdrental_test_restore;
```

In **pgAdmin** (connected to dvdrental):

```
UPDATE customer SET last_name = 'Smith' WHERE customer_id = 1;
```

Key discussion point: This is why backups must be taken **regularly**. Any data changed between the last backup and a crash is **lost** — unless you have WAL archiving and Point-In-Time Recovery (Week 9 topic).

Practice — Part D (Full Lab)

Complete the full backup & restore cycle.

STEP 1: Verify current state — in **pgAdmin** (connected to dvdrental):

```
SELECT COUNT(*) AS total_customers FROM customer;

SELECT COUNT(*) AS total_rentals FROM rental;

SELECT COUNT(*) AS total_payments FROM payment;

-- Write down these numbers!
```

STEP 2: Take a full backup — in **PowerShell**:

```
● ● ●

pg_dump -U postgres -f student_backup.sql dvdrental

# Verify the file was created

dir student_backup.sql
```

STEP 3: Create a new database — in **pgAdmin** (connected to postgres database):

```
CREATE DATABASE my_restored_db;
```

Restore — in **PowerShell**:

```
● ● ●

psql -U postgres -d my_restored_db -f student_backup.sql
```

STEP 4: Verify the restore — in **pgAdmin** (connect to my_restored_db):

```
SELECT COUNT(*) AS total_customers FROM customer;

SELECT COUNT(*) AS total_rentals FROM rental;

SELECT COUNT(*) AS total_payments FROM payment;
```

```
-- Counts should match what you wrote down!
```

STEP 5: Verify indexes were restored too:

```
SELECT indexname FROM pg_indexes
WHERE schemaname = 'public' ORDER BY indexname LIMIT 10;
```

STEP 6: Clean up — in **pgAdmin** (connected to `postgres` database):

```
DROP DATABASE IF EXISTS my_restored_db;
```

Answers / Expected results:

- Step 4: Row counts in `my_restored_db` should **exactly match** the original `dvdrental`. `pg_dump` captures all data.
- Step 5: Indexes are included in the backup — `pg_dump` captures table structure, indexes, constraints, sequences, and data.
- **If counts don't match:** Someone may have modified `dvdrental` between the backup and the restore. The backup is a point-in-time snapshot.

Common Questions — Restore

Q: Can I restore a backup into an existing database that already has data?

You can, but you'll get errors — the tables already exist, primary keys will conflict. It's best practice to always restore into a **new, empty database**. If you need to replace an existing database, drop it first, recreate it, then restore.

Q: Can I restore a backup to a different PostgreSQL version?

Yes — this is one of the main advantages of logical backups (`pg_dump`). Because the backup is just SQL statements, it works across PostgreSQL versions. You can dump from PostgreSQL 14 and restore into PostgreSQL 16. Physical backups don't have this portability.

Q: What if the restore fails halfway through?

The restored database will be incomplete — some tables will have data, others won't. You should drop the partially restored database, fix the problem (usually disk space or permissions), and restore again from scratch.